

Axiomander Design Review: Evolving ImpIR into a Semantically Robust Verification IR

Status

Recommendation

Executive Summary

The current ImpIR is already a useful verification-oriented intermediate language. It supports:

- mutable state
- heap operations
- exceptions
- contracts
- nondeterministic specification calls
- weakest precondition reasoning

However, several aspects of the current design were chosen for convenience rather than semantic precision.

These choices will eventually limit:

- proof soundness
- contract expressiveness
- concurrency support
- resource reasoning
- integration with Iris or other separation logics
- verification of databases, queues, and distributed systems

This document proposes a sequence of improvements that strengthen ImpIR regardless of whether Iris is ultimately adopted.

The guiding principle is:

ImpIR should become the semantic source of truth for Axiomander.

Current Strengths

Current ImpIR already contains several good design decisions.

Explicit State

```
Record state := mkState {  
  ls : var -> value;  
  hs : (var * var) -> value  
}.
```

The distinction between locals and heap-like state is valuable.

Explicit Outcomes

```
OReturn  
ORaise
```

This is significantly better than encoding exceptions as special values.

The outcome-based WP calculus is already pointing in the right direction.

Contract-Aware Calls

```
CCall
```

provides an effective mechanism for reasoning about partially specified functions.

This should remain part of the architecture.

Issue 1: Incorrect Try/Except Semantics

Problem

Current operational semantics execute the exception handler even when the body returns normally.

This contradicts:

- Python semantics
- the current WP calculus
- user expectations

Recommendation

Normal return should bypass the handler entirely.

Desired behavior:

```
try:  
    body  
except:  
    handler
```

becomes:

```
body returns  
    -> handler skipped  
  
body raises  
    -> handler executed
```

Priority

Critical.

This should be fixed immediately.

Issue 2: Total Coercions Hide Errors

Current helper functions contain behavior such as:

```
asZ(non_integer) = 0  
asString(non_string) = ""
```

This is useful for automation but dangerous for semantics.

Example:

```
1 + "abc"
```

should not silently become:

```
1 + 0
```

inside the verifier.

Risks

Silent coercions:

- mask bugs
- weaken specifications
- produce misleading proofs

Recommendation

Introduce explicit evaluation failure.

Instead of:

```
eval_expr -> value
```

use:

```
eval_expr -> result value
```

where:

```
Success value  
Failure runtime_error
```

This immediately improves semantic fidelity.

Issue 3: Separate Program Semantics from Verification Convenience

Current ImpIR combines:

```
program behavior  
verification shortcuts
```

into a single language.

Examples:

```
CAssume  
contract-bearing CCall  
havoc
```

These are verifier constructs rather than executable semantics.

Recommendation

Split concepts into:

Executable Core

```
assignment  
heap operations  
conditionals  
loops  
calls  
exceptions
```

Verification Layer

```
assume  
havoc  
trusted calls  
proof debt  
contract summaries
```

This separation will greatly simplify future reasoning.

Issue 4: Heap Representation

Current heap:

```
(var * var) -> value
```

is a total map.

Every field effectively exists forever.

Problems

Cannot distinguish:

```
field absent  
field present with None
```

Ownership becomes awkward.

Resource reasoning becomes awkward.

Future database modeling becomes awkward.

Recommendation

Use finite maps.

Example:

```
gmap heap_key value
```

where:

```
heap_key :=  
  object_id * field_name
```

Benefits:

- absence becomes meaningful
- ownership becomes meaningful
- framing becomes meaningful

Even without Iris this is preferable.

Issue 5: Introduce Explicit Resource Regions

Current ImpIR treats all heap state uniformly.

Future applications require distinct resources:

```
Python heap
database
Kafka
Azure Queue
Redis
filesystem
cache
```

Recommendation

Introduce logical regions.

Example:

```
RegionHeap
RegionDatabase
RegionQueue
RegionFilesystem
```

or:

```
Region(name)
```

State becomes:

```
State =
  locals
  heap
  resource regions
```

This prepares the system for:

- ownership
- framing
- concurrency

- distributed systems

without requiring Iris immediately.

Issue 6: Calls Should Separate Semantics from Contracts

Current:

```
CCall(name, args, pre, post, writes, target)
```

mixes:

```
execution  
specification  
verification
```

into one construct.

Recommendation

Separate:

Executable Call

```
Call(f, args)
```

Contract Summary

```
Spec(f)
```

Trusted External Transition

```
ExternalCall(resource_transition)
```

This keeps operational semantics cleaner.

Issue 7: Introduce Small-Step Semantics

Current semantics are big-step.

Big-step is excellent for:

```
WP generation
symbolic reasoning
SMT obligations
```

but weak for:

```
exceptions
concurrency
interleavings
resource ownership
```

Recommendation

Retain big-step semantics.

Add small-step semantics alongside them.

Both should coexist.

Desired structure:

```
BigStep
  for verification automation

SmallStep
  for semantic foundations
```

Then prove consistency between them.

Issue 8: Explicit Runtime Errors

Currently ImpIR primarily distinguishes:

```
Return
Raise
```

Future reasoning benefits from:

```
Return
Raise
RuntimeError
```

Examples:

```
divide by zero
invalid index
invalid attribute access
invalid coercion
```

This improves diagnostic quality and proof precision.

Issue 9: Introduce a Core Language

ImpIR is becoming large.

Recommendation:

```
ImpCore
```

containing:

```
expressions
assignment
heap access
conditionals
loops
exceptions
calls
```

Everything else becomes elaboration.

Examples:

```
havoc
assume
trusted calls
contract summaries
```

desugar into ImpCore.

Benefits:

- cleaner semantics
- easier proofs
- easier backend generation

Issue 10: Future-Proof Exception Semantics

Current exception support is promising.

However exceptions should become first-class semantic entities.

Eventually the language should model:

```
raise
try
except
finally
```

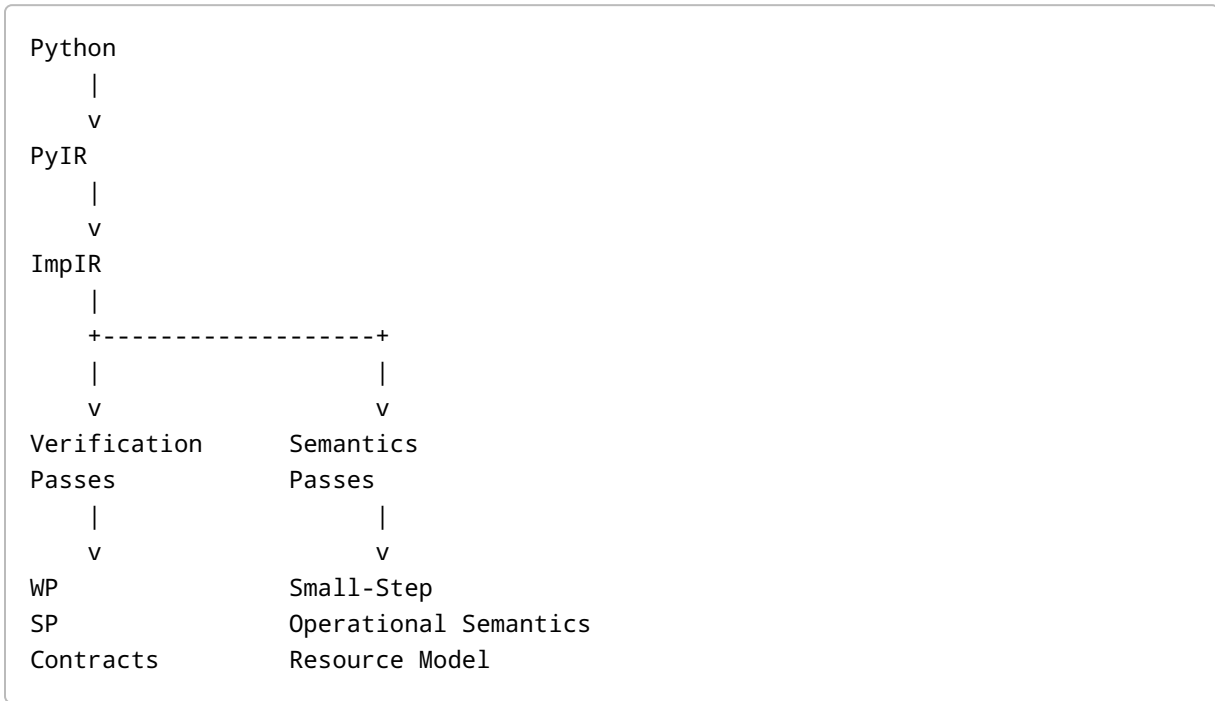
directly.

Contracts should support:

```
Raises(ValueError)
Raises(KeyError)
```

without encoding exceptions as values.

Proposed Architecture



Immediate Implementation Priorities

Priority 1

- Fix try/except semantics.

Priority 2

- Remove silent coercions.
- Introduce explicit evaluation failures.

Priority 3

- Replace total heap maps with finite maps.

Priority 4

- Separate executable semantics from verifier constructs.

Priority 5

- Introduce resource regions.

Priority 6

- Define a small-step semantics alongside the existing big-step semantics.

Priority 7

- Create ImpCore as a minimal semantic kernel.
-

Long-Term Goal

ImpIR should evolve from:

verification-oriented imperative language

into:

a rigorous semantic model of Python-relevant computation

capable of supporting:

- WP reasoning
- SP reasoning
- contract checking
- concurrency
- database verification
- queue verification
- resource ownership
- theorem proving backends
- Iris integration

without requiring major redesign later.